



Product & Technical Overview

Visual Node-Based Canvas Application

v0.1.0 | June 2026 | Xavier Garès | GPLv3

1. What Is theFlow!

theFlow! is a cross-platform desktop application for organising images, video, audio, documents, text, web pages, and PDF files in a freeform infinite canvas. Content is represented as nodes connected by Bézier curves. The result is a visual workspace that feels closer to a physical desk or mood board than a traditional file manager or note-taking app.

The app is written in Python (PyQt6) and distributed as self-contained native binaries for macOS, Windows, and Linux — no installation of Python or any dependency is required on end-user machines.

Key Capabilities

- Infinite 2D canvas with smooth zoom and pan
- 9 node types: Text, Image, Movie, Audio, Document, Paint, Web, PDF, Dot
- Inline media viewers that expand below each node (video player, audio waveform, PDF viewer, web screenshot)
- Bézier connection curves linking nodes to express relationships
- Backdrop grouping — moves all pinned child nodes together
- Canvas annotation — freehand drawing overlay with color, thickness, and style
- Full undo/redo, copy/paste, import/export of node subsets
- Light and dark themes with live propagation to all canvas items
- .flow file format — JSON, human-readable, forward-compatible
- File association: double-clicking a .flow file opens theFlow! on all platforms

2. Technology Stack

Layer	Technology	Notes
Language	Python 3.9–3.12	3.9 pinned for Linux; 3.12 for macOS/Windows
UI Framework	PyQt6 (Qt6 bindings)	QGraphicsScene/View for canvas; Qt Multimedia for audio/video
Media Playback	Qt Multimedia + FFmpeg	Bundled FFmpeg backend; AVFoundation on macOS
Packaging	PyInstaller 6.x	Freezes Python + Qt into single binary per platform
macOS Installer	DMG (hdiutil)	Drag-to-Applications bundle
Windows Installer	Inno Setup 6	Standard .exe installer with registry file association
Linux Installer	dpkg-deb / rpmbuild	.deb (Ubuntu/Debian) and .rpm (Fedora/RHEL)
Persistence	JSON (.flow files)	Human-readable; base64 for embedded binary data

3. Architecture

theFlow! has no package hierarchy — all modules are flat Python files in the project root. The main structural components are:

Module	Responsibility
main.py	App entry point. Platform-specific QApplication subclass, settings load, file-open event routing
view_logic.py	QGraphicsView subclass. All user input: pan/zoom, keyboard shortcuts, node creation, file I/O
scene_logic.py	QGraphicsScene subclass. Serialisation (to_dict/from_dict), undo/redo stack, backdrop child management
node.py	All content node types and their inline media viewers
curve.py	Bézier ConnectionLine and Socket
backdrop.py / note.py	Backdrop grouping and Sticky Note with resize handles
paint_on_canvas.py	Canvas freehand annotation (CanvasPainter, CanvasStroke, StrokeToolbar)
menu.py / ui_components.py	Right-click context menu and Tab creation menu
settings.py / config.py / utils.py	Settings dialog, global constants, shared helpers

Canvas Model

The canvas uses Qt's QGraphicsScene + QGraphicsView model. Scene coordinates are infinite; the view applies a smooth scale transform for zoom. Inline media viewers (video, audio, PDF, web) are QWidget instances parented to the viewport — not the scene — and are repositioned manually on every scroll or zoom event. This approach supports Qt multimedia widgets that cannot be embedded directly in the scene graph.

4. Node Types

Node	Key	What It Does	Inline Viewer
Text	T	Rich text with bold/italic/underline, hyperlinks	Scrollable QTextBrowser
Image	I	PNG, JPG, GIF, WebP, SVG	Scaled image with resize grip
Movie	M	MP4, MOV, AVI, MKV	Video player with click-to-seek timeline
Audio	A	MP3, WAV, AAC, FLAC	RMS waveform + seek slider + playback controls
Document	D	PDF via QPdfDocument	Scrollable PDF viewer, FitToWidth
Paint	P	Pixel paint canvas (QPixmap)	Resizable embedded canvas with brushes
Dot	Q	Connection pass-through point	None

All nodes share: selectable/movable canvas item, input + output sockets, shape options (rectangle, rounded, ellipse, diamond, hexagon), color/font customisation, double-click edit dialog, context menu, and undo/redo participation.

Inline viewers are opened with ↓ and closed with ↑. Shift+↓ expands all selected nodes; Shift+↑ collapses them.

5. The .flow File Format

Workspaces are saved as UTF-8 JSON with a .flow extension. The format is intentionally simple and forward-compatible — unknown keys are ignored, so files created by newer versions open in older ones without errors.

Key	Contents
nodes	Array of node objects — type, position, name, color, shape, and type-specific fields (path, URL, base64 image data, etc.)
connections	Array of {from, to} socket ID pairs encoded as strings
strokes	Array of freehand annotation strokes — color, width, style, point arrays

Binary content (paint canvases, web thumbnails, exported images) is stored as base64-encoded PNG within the JSON. File paths for media nodes (video, audio, PDF) are stored as absolute paths — users should keep media files alongside their .flow workspace.

Import / Export

Export Selected (Ctrl+E) saves only the selected nodes and their interconnecting curves as a standalone .flow file. Import Nodes (Ctrl+I) merges a .flow file into the current canvas without clearing it — imported nodes are offset slightly to avoid exact overlap.

6. Build System

Each platform has its own build script that handles the full pipeline from source to distributable in a single command.

Platform	Command	Output
macOS	<code>./build_mac.sh</code>	<code>dist/theFlow.app</code> + <code>dist/theFlow-0.1.0.dmg</code>
Windows	<code>build_windows.bat</code>	<code>dist\theFlow.exe</code> + <code>dist\theFlow-0.1.0-setup.exe</code>
Linux DEB	<code>./build_deb.sh</code>	<code>dist/thewflow</code> + <code>dist/thewflow_0.1.0_amd64.deb</code>
Linux RPM	<code>./build_rpm.sh</code>	<code>dist/thewflow</code> + <code>dist/thewflow-0.1.0-1.x86_64.rpm</code>

PyInstaller bundles the Python interpreter, all app modules, Qt6 shared libraries, and the FFmpeg multimedia backend into the binary. No Python installation is needed on end-user machines.

The macOS build script additionally converts the app icon to .icns format, code-signs the bundle with an ad-hoc signature, and wraps it in a DMG with an Applications symlink for drag-install. The Windows build runs Inno Setup after PyInstaller to produce the .exe installer which registers the .flow file association in the Windows registry.

7. Platform Differences

Feature	macOS	Windows	Linux
Python	3.12 (venv)	3.12	3.9 (pinned)
Web Node viewer	Playwright + WebEngine float	Playwright only	Playwright only
Exit method	os._exit() (avoids Qt destructor crash)	sys.exit()	sys.exit()
UPX compression	Í	Í	φ (corrupts FFmpeg)
Settings path	Next to .app / dev root	Next to .exe	~/.config/theflow/
File open event	QFileOpenEvent (Finder)	sys.argv[1] (Explorer)	sys.argv[1] (file manager)
Installer type	DMG	.exe (Inno Setup)	.deb or .rpm

8. Key Design Decisions

Viewport-Parented Inline Viewers

Media viewers (video, PDF, audio waveform) are plain QWidget instances parented to the viewport, not embedded in the scene graph. This is necessary because Qt multimedia widgets (QVideoWidget, QPdfView) do not work correctly inside QGraphicsProxyWidget. The reposition() pattern — recalculating pixel coordinates on every scroll/zoom — is verbose but reliable across all platforms.

Playwright for Web Screenshots

Web page rendering uses Playwright/Chromium running in a background thread, not Qt WebEngine embedded in the binary. This keeps the distributed binary small and avoids AVFoundation compositor conflicts that arise on macOS when WebEngine is bundled via PyInstaller. Users must run ``playwright install chromium`` once; the browser cache persists across app updates.

JSON File Format

Plain JSON was chosen over SQLite or a binary format for simplicity, human-readability, and version control compatibility. .flow files diff cleanly in git. Base64 encoding is used only where binary blobs are unavoidable (embedded paint canvas, web thumbnail cache).

Flat Module Structure

All Python modules are flat files in the project root — no sub-packages. This eliminates PyInstaller packaging complexity with package `__init__.py` imports and keeps the module namespace simple. The tradeoff is a longer file list in the project root as the codebase grows.